

Implementation details for the 4D MFEM branch

Patrick Saber

*Advised by David M. Williams, Associate Professor
The Pennsylvania State University
Department of Mechanical Engineering*

Overview

Overview of the Presentation

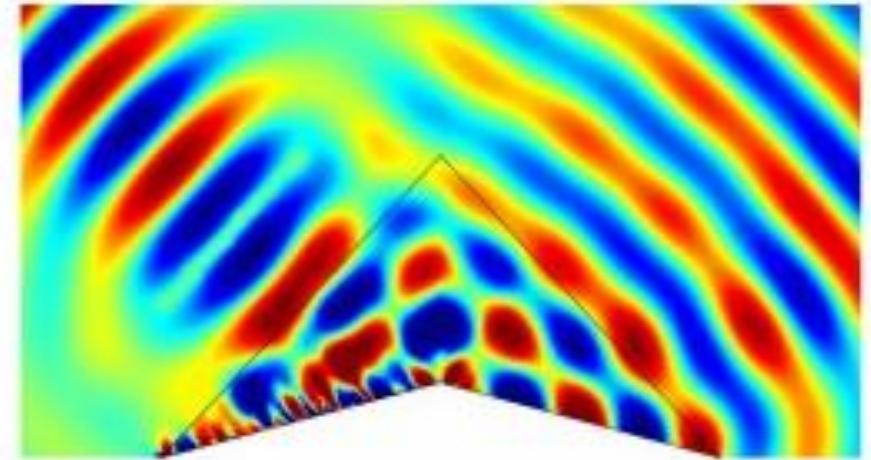
- I. Motivation
- II. Mathematical Preliminaries
 - i. Vector Spaces and Derivative Operators
 - ii. Conforming Shape Functions
- III. Implementation Details
 - i. Degrees of Freedom
 - ii. Generalized Vandermonde Matrices
 - iii. Orientation Matching
- IV. Order of Accuracy Results
- V. Final Thoughts

I. Motivation

Maxwell's Equations

Inhomogeneous Form of Maxwell's equations (3-Dimensional)

$$\begin{aligned}\nabla \cdot E &= 4\pi\rho, & \nabla \cdot B &= 0, \\ c\nabla \times E + \frac{\partial}{\partial t}B &= 0, & c\nabla \times B - \frac{\partial}{\partial t}E &= 4\pi J,\end{aligned}$$



Jichun Li. Recent progress on mathematical analysis and numerical simulations for Maxwell's equations in perfectly matched layers and complex media: a review[J]. *Electronic Research Archive*, 2024, 32(3): 1901-1922. doi: [10.3934/era.2024087](https://doi.org/10.3934/era.2024087)

Motivation

4D Formulation of Inhomogeneous Maxwell's Equations

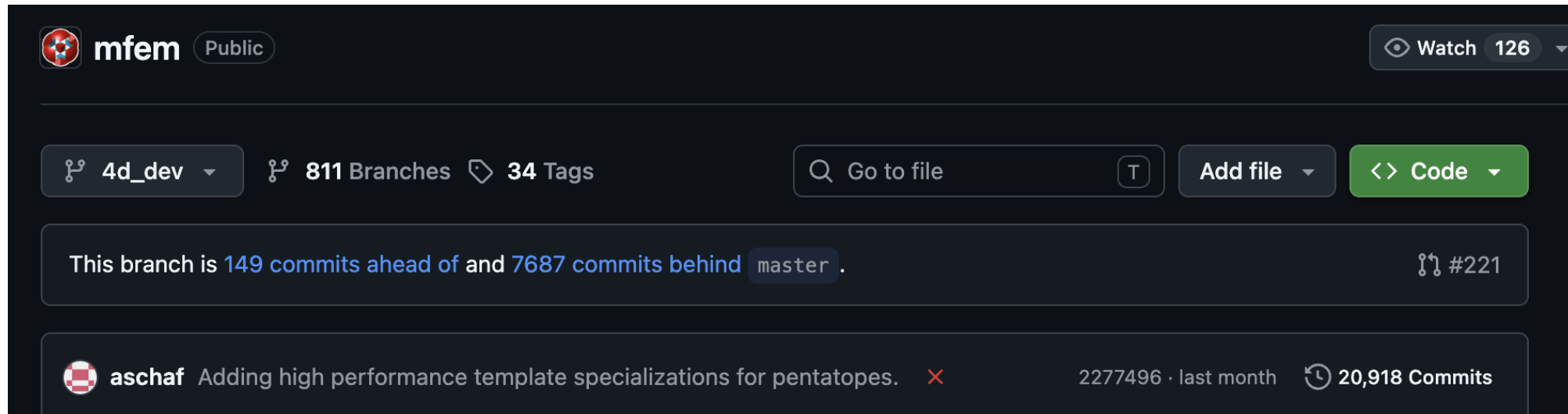
$$F = \frac{1}{2} \begin{bmatrix} 0 & -cB_x & -cB_y & -cB_z \\ cB_x & 0 & -E_z & E_y \\ cB_y & E_z & 0 & -E_x \\ cB_z & -E_y & E_x & 0 \end{bmatrix}, \quad H = \frac{1}{2} \begin{bmatrix} 0 & -cE_x & -cE_y & -cE_z \\ cE_x & 0 & B_z & -B_y \\ cE_y & -B_z & 0 & B_x \\ cE_z & B_y & -B_x & 0 \end{bmatrix}, \quad G = - \begin{bmatrix} \rho \\ j_x \\ j_y \\ j_z \end{bmatrix}$$

- Final equations

$$\text{curl}(F) = 4\pi G, \quad \text{curl}(H) = 0, \quad \text{div}(G) = 0$$

Motivation

- **Objective:** high-order, conforming finite elements for Maxwell's equations (4D)
- **Problem:** existing 4D branch of MFEM primarily contains low-order finite elements
- **Current developers:** Patrick Saber (Penn State), David M. Williams (Penn State), Tamas Horvath (Oakland University), and ??



II. Mathematical Preliminaries

Vector Spaces and Derivative Operators in 4D

8

4D Derivative Operators

grad, skwGrad, curl, div.

Infinite-Dimensional Sobolev Spaces

$$\begin{aligned} H(\text{grad}, \Omega, \mathbb{R}) &= \{u \in L^2(\Omega, \mathbb{R}) : \text{grad } u \in L^2(\Omega, \mathbb{R}^4)\}, \\ H(\text{skwGrad}, \Omega, \mathbb{R}^4) &= \{E \in L^2(\Omega, \mathbb{R}^4) : \text{skwGrad } E \in L^2(\Omega, \mathbb{K})\}, \\ H(\text{curl}, \Omega, \mathbb{K}) &= \{F \in L^2(\Omega, \mathbb{K}) : \text{curl } F \in L^2(\Omega, \mathbb{R}^4)\}, \\ H(\text{div}, \Omega, \mathbb{R}^4) &= \{G \in L^2(\Omega, \mathbb{R}^4) : \text{div } G \in L^2(\Omega, \mathbb{R})\}, \\ L^2(\Omega, \mathbb{R}) &= \left\{ q : \int_{\Omega} q^2 dx < \infty \right\} \end{aligned}$$

de Rham Complex in 4D

9

- C^∞ de Rham complex in 4D

$$C^\infty(\Omega, \mathbb{R}) \xrightarrow{\text{grad}} C^\infty(\Omega, \mathbb{R}^4) \xrightarrow{\text{skwGrad}} C^\infty(\Omega, \mathbb{K}) \xrightarrow{\text{curl}} C^\infty(\Omega, \mathbb{R}^4) \xrightarrow{\text{div}} C^\infty(\Omega, \mathbb{R})$$

- L^2 de Rham complex in 4D

$$H(\text{grad}, \Omega, \mathbb{R}) \xrightarrow{\text{grad}} H(\text{skwGrad}, \Omega, \mathbb{R}^4) \xrightarrow{\text{skwGrad}} H(\text{curl}, \Omega, \mathbb{K}) \xrightarrow{\text{curl}} H(\text{div}, \Omega, \mathbb{R}^4) \xrightarrow{\text{div}} L^2(\Omega, \mathbb{R}).$$

Derivative Operators in 4D

- New derivative operators $u \in L^2(\Omega, \mathbb{R})$, $E \in L^2(\Omega, \mathbb{R}^4)$

$$\begin{aligned} [\text{grad } u]_i &= \partial_i u, \\ [\text{Grad } E]_{ij} &= \partial_j E_i \quad i = 1, \dots, 4, \quad j = 1, \dots, 4 \end{aligned}$$

$$[\text{skwGrad } E] = \frac{1}{2} \left([\text{Grad } E]^T - [\text{Grad } E] \right)$$

Derivative Operators in 4D

11

- New derivative operators $F \in L^2(\Omega, \mathbb{K})$, $G \in L^2(\Omega, \mathbb{R}^4)$

$$[\text{curl } F]_i = \sum_{k,l=1}^4 \varepsilon_{ijkl} \partial_j F_{kl}$$

$$[\text{div } G] = \partial_i G_i$$

Auxiliary Derivative Operators in 4D

12

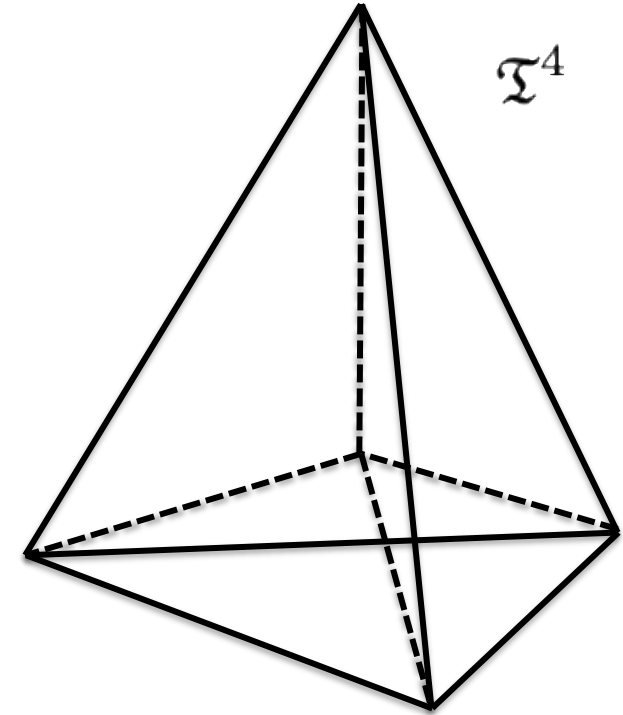
- New auxiliary derivative operators $F \in L^2(\Omega, \mathbb{K}), E \in L^2(\Omega, \mathbb{R}^4)$

$$[\text{Curl } E]_{ij} = \sum_{k,l=1}^4 \varepsilon_{ijkl} \partial_k E_l$$

$$[\text{Div } F]_i = \partial_j F_{ij}$$

Finite Element of Interest: the Pentatope

Geometry Type	Pentatope
Vertices	5
Edges	10
Triangular Planes	10
Tetrahedral Facets	5



Finite-Dimensional Subspaces on the Pentatope

$$V_k \Lambda^0(\mathfrak{T}^4) := P^k(\mathfrak{T}^4),$$

H(grad)-conforming

$$V_k \Lambda^1(\mathfrak{T}^4) := (P^{k-1}(\mathfrak{T}^4))^4 \oplus \left\{ p \in (\tilde{P}^k(\mathfrak{T}^4))^4 \mid p \cdot x = 0 \right\},$$

H(skewGrad)-conforming

$$V_k \Lambda^2(\mathfrak{T}^4) := \mathcal{L}((P^{k-1}(\mathfrak{T}^4))^6) \oplus \left\{ B \in \mathcal{L}((\tilde{P}^k(\mathfrak{T}^4))^6) \mid Bx = 0 \right\}$$

H(curl)-conforming

$$V_k \Lambda^3(\mathfrak{T}^4) := (P^{k-1}(\mathfrak{T}^4))^4 \oplus \tilde{P}^{k-1}(\mathfrak{T}^4)x,$$

H(div)-conforming

$$V_k \Lambda^4(\mathfrak{T}^4) := P^{k-1}(\mathfrak{T}^4).$$

L2-conforming

P^k is the polynomial space of degree $\leq k$,

$\tilde{P}^k$ is the homogeneous polynomial space of degree k

Finite-Dimensional Subspaces on the Pentatope: H(curl)-conforming

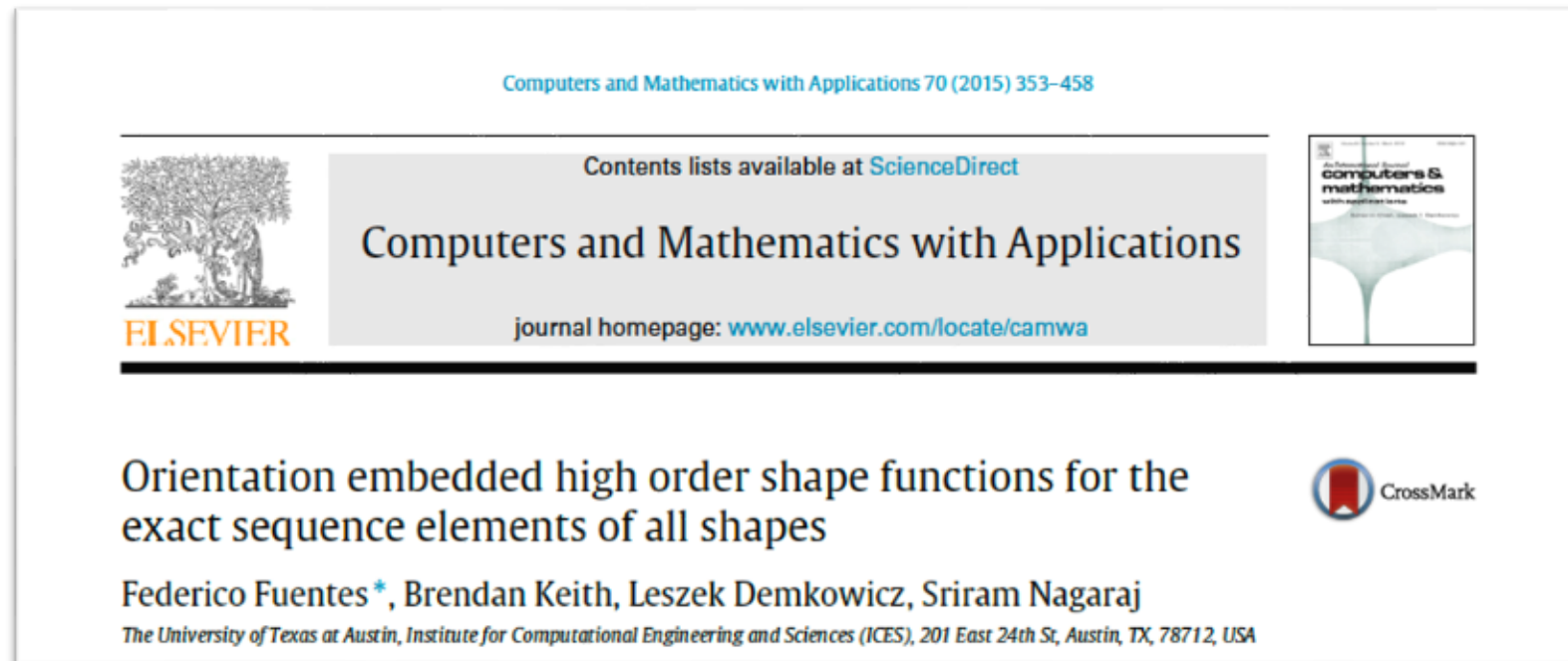
$$V_k \Lambda^2(\mathfrak{T}^4) := \mathcal{L}((P^{k-1}(\mathfrak{T}^4))^6) \oplus \left\{ B \in \mathcal{L}((\tilde{P}^k(\mathfrak{T}^4))^6) \mid Bx = 0 \right\}.$$

$$\mathcal{L} \cdot : \mathbb{R}^6 \rightarrow \mathbb{K} : \mathcal{L} \begin{bmatrix} w_{12} \\ w_{13} \\ w_{14} \\ w_{23} \\ w_{24} \\ w_{34} \end{bmatrix} := \begin{bmatrix} 0 & w_{12} & w_{13} & w_{14} \\ -w_{12} & 0 & w_{23} & w_{24} \\ -w_{13} & -w_{23} & 0 & w_{34} \\ -w_{14} & -w_{24} & -w_{34} & 0 \end{bmatrix}.$$

Inspiration for Conforming Shape Functions

16

- Our 4D approach is inspired by the approach of Fuentes et al. in 3D
- F. Fuentes, B. Keith, L. Demkowicz, and S. Nagaraj, “Orientation embedded high order shape functions for the exact sequence elements of all shapes,” *Computers and Mathematics with Applications*, (2015)



Conforming Shape Functions in 3D

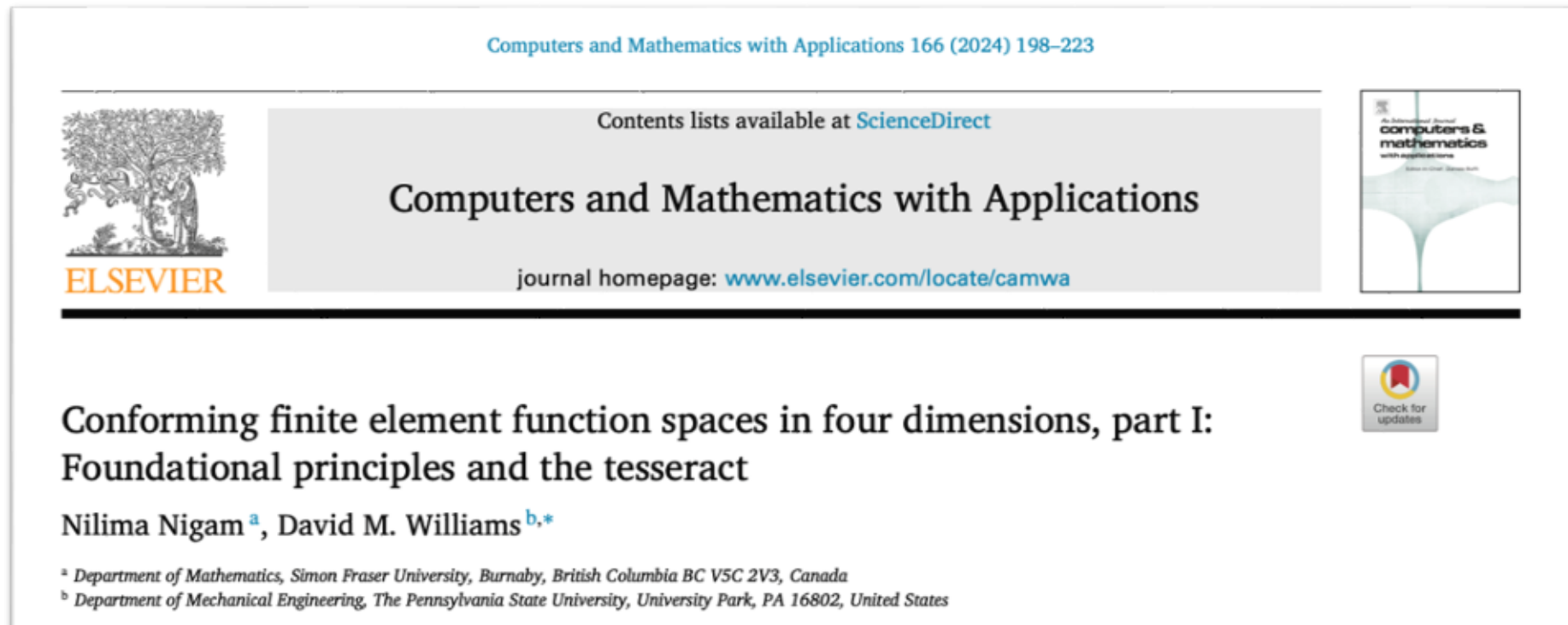
17

- Advantages of the Fuentes et al. approach
 - Intuitive construction based on barycentric coordinates
 - Easily generalizable to higher dimensions
 - Flexibility for representing the basis functions of multiple derivative operators with relatively small modifications to the formulation
 - Hierarchical structure
 - Previously implemented in MFEM for square pyramids in 3D by M. Stowell
- Extended to pentatopes, tesseracts, and tetrahedral prisms by Nigam and Williams

Conforming Shape Functions in 4D

18

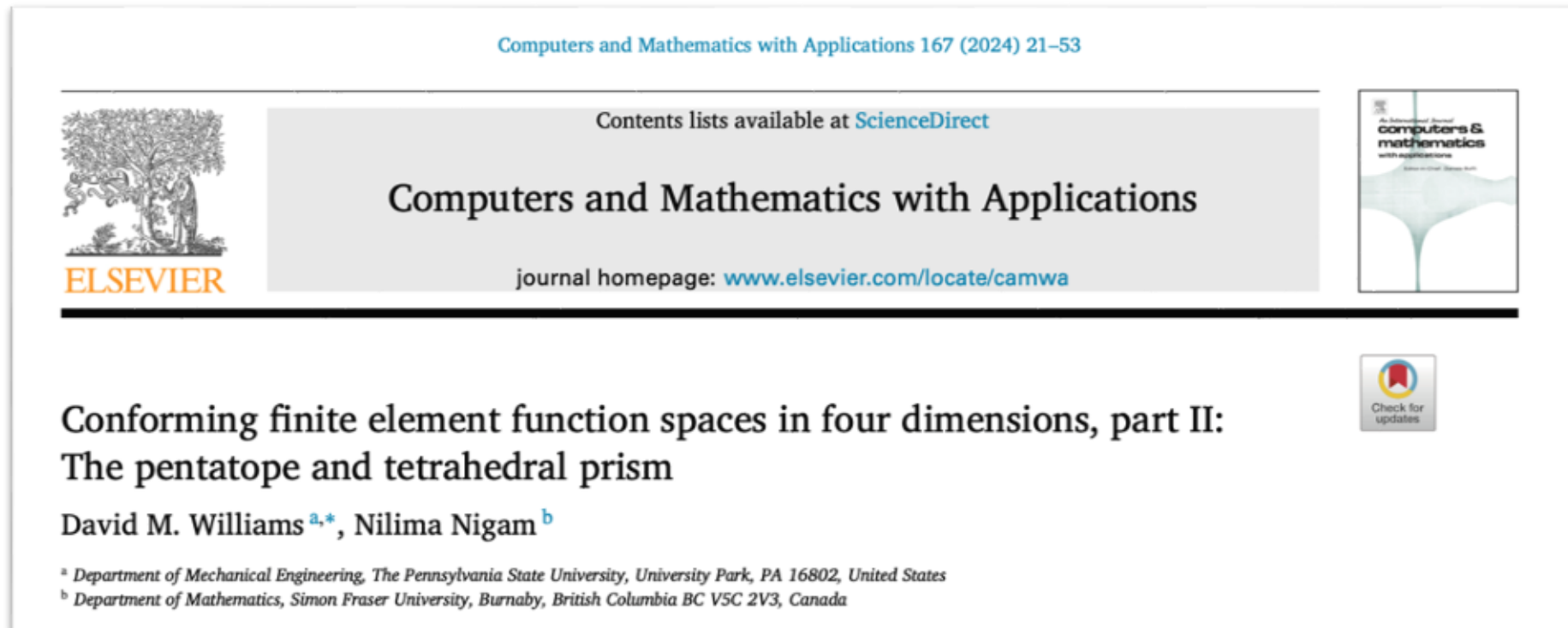
- D. Williams, N. Nigam, “Conforming Finite Element Function Spaces in Four Dimensions, Part I: Foundational principles and the tesseract,” *Computers and Mathematics with Applications*, (2024)



Conforming Shape Functions in 4D

19

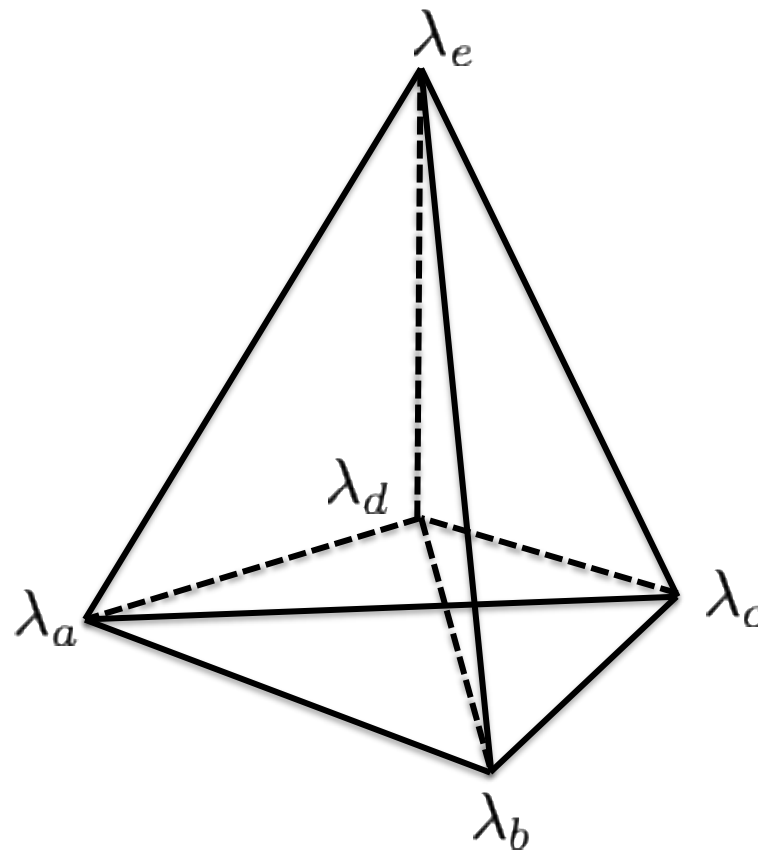
- D. Williams, N. Nigam, “Conforming Finite Element Function Spaces in Four Dimensions, Part II: The Pentatope and Tetrahedral Prism,” *Computers and Mathematics with Applications*, (2024)



Conventions on the Pentatope

20

- Barycentric coordinates are associated with each vertex of the pentatope
- Vertices are a , b , c , d , and e



H(grad)-Conforming Shape Functions

- We obtain high-order functions by multiplying together the appropriate polynomial functions
- Polynomial functions used for H(grad)-conforming
 - Integrated Legendre polynomials
 - Integrated Jacobi polynomials

$$\phi_{ij\ell m} \left(\vec{\lambda}_{abcde}(x) \right) = L_i(\lambda_b; \lambda_a + \lambda_b) L_j^{2i}(\lambda_c; \lambda_a + \lambda_b + \lambda_c) \\ \cdot L_\ell^{2(i+j)}(\lambda_d; \lambda_a + \lambda_b + \lambda_c + \lambda_d) L_m^{2(i+j+\ell)}(\lambda_e)$$

H(skwGrad)-Conforming Shape Functions

- We obtain high-order functions by scaling the Whitney form by appropriate polynomial functions
- Polynomial functions used for H(skwGrad)-conforming
 - Legendre polynomials
 - Integrated Jacobi polynomials

Whitney form:

$$(\lambda_a \nabla \lambda_b - \lambda_b \nabla \lambda_a)$$

$$E_{ij\ell m}^r \left(\vec{\lambda}_{abcde}(x) \right) = P_i(\lambda_b; \lambda_a + \lambda_b) L_j^{2i+1}(\lambda_c; \lambda_a + \lambda_b + \lambda_c) \\ \cdot L_\ell^{2(i+j)}(\lambda_d; \lambda_a + \lambda_b + \lambda_c + \lambda_d) L_m^{2(i+j+\ell)}(\lambda_e) (\lambda_a \nabla \lambda_b - \lambda_b \nabla \lambda_a),$$

H(curl)-Conforming Shape Functions

- Polynomial functions used for H(curl)-conforming
 - Legendre polynomials
 - Jacobi polynomials
 - Integrated Jacobi polynomials

Whitney form:

$$\left[\begin{aligned} &\lambda_a (\nabla \lambda_b \otimes \nabla \lambda_c - \nabla \lambda_c \otimes \nabla \lambda_b) \\ &+ \lambda_b (\nabla \lambda_c \otimes \nabla \lambda_a - \nabla \lambda_a \otimes \nabla \lambda_c) + \lambda_c (\nabla \lambda_a \otimes \nabla \lambda_b - \nabla \lambda_b \otimes \nabla \lambda_a) \end{aligned} \right]$$

$$V_{ij\ell m}^r \left(\vec{\lambda}_{abcde}(x) \right) =$$

$$P_i(\lambda_b; \lambda_a + \lambda_b) P_j^{2i+1}(\lambda_c; \lambda_a + \lambda_b + \lambda_c) L_\ell^{2(i+j+1)}(\lambda_d; \lambda_a + \lambda_b + \lambda_c + \lambda_d) L_m^{2(i+j+\ell)}(\lambda_e)$$

$$\cdot \left[\lambda_a (\nabla \lambda_b \otimes \nabla \lambda_c - \nabla \lambda_c \otimes \nabla \lambda_b) + \lambda_b (\nabla \lambda_c \otimes \nabla \lambda_a - \nabla \lambda_a \otimes \nabla \lambda_c) + \lambda_c (\nabla \lambda_a \otimes \nabla \lambda_b - \nabla \lambda_b \otimes \nabla \lambda_a) \right]$$

H(div)-Conforming Shape Functions

- Polynomial functions used for H(div)-conforming
 - Legendre polynomials
 - Jacobi polynomials
 - Integrated Jacobi polynomials

Whitney form:

$$\left[\begin{aligned} &\lambda_a (\nabla \lambda_b \times \nabla \lambda_c \times \nabla \lambda_d) - \lambda_b (\nabla \lambda_c \times \nabla \lambda_d \times \nabla \lambda_a) \\ &+ \lambda_c (\nabla \lambda_d \times \nabla \lambda_a \times \nabla \lambda_b) - \lambda_d (\nabla \lambda_a \times \nabla \lambda_b \times \nabla \lambda_c) \end{aligned} \right]$$

$$\psi_{ij\ell m}^r \left(\vec{\lambda}_{abcde}(x) \right) =$$

$$P_i(\lambda_b; \lambda_a + \lambda_b) P_j^{2i+1}(\lambda_c; \lambda_a + \lambda_b + \lambda_c) P_\ell^{2(i+j+1)}(\lambda_d; \lambda_a + \lambda_b + \lambda_c + \lambda_d)$$

$$\cdot L_m^{2(i+j+\ell)+3}(\lambda_e) \left[\begin{aligned} &\lambda_a (\nabla \lambda_b \times \nabla \lambda_c \times \nabla \lambda_d) - \lambda_b (\nabla \lambda_c \times \nabla \lambda_d \times \nabla \lambda_a) \\ &+ \lambda_c (\nabla \lambda_d \times \nabla \lambda_a \times \nabla \lambda_b) - \lambda_d (\nabla \lambda_a \times \nabla \lambda_b \times \nabla \lambda_c) \end{aligned} \right]$$

L2-Conforming Shape Functions

- We obtain high-order functions by multiplying together the appropriate polynomial functions
- Polynomial functions used for L2-conforming
 - Legendre polynomials
 - Jacobi polynomials

$$\mathcal{V}_{ij\ell m} \left(\vec{\lambda}_{abcde}(x) \right) = P_i(\lambda_b; \lambda_a + \lambda_b) P_j^{2i+1}(\lambda_c; \lambda_a + \lambda_b + \lambda_c) \\ \cdot P_\ell^{2(i+j+1)}(\lambda_d; \lambda_a + \lambda_b + \lambda_c + \lambda_d) P_m^{2(i+j+\ell)+3}(\lambda_e)$$

III. Implementation Details

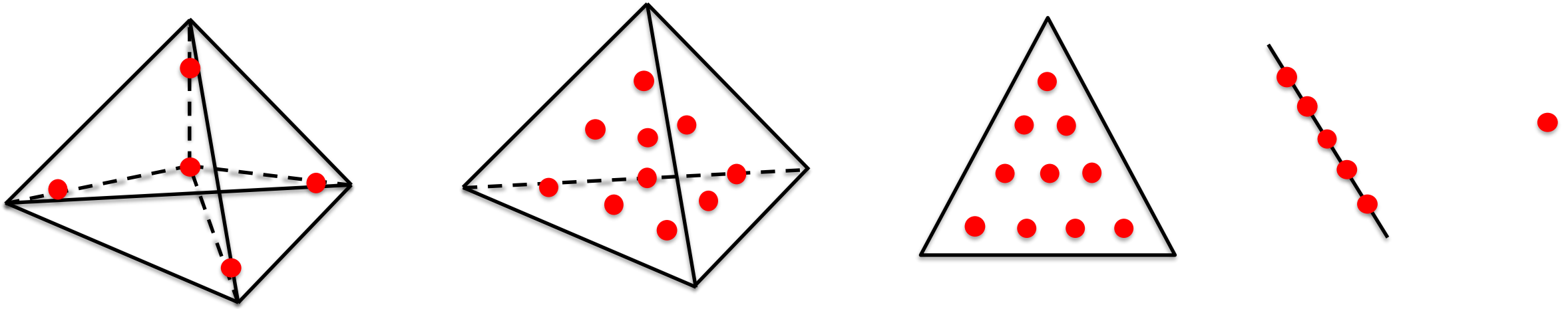
Degrees of Freedom Summary

27

	H(grad)	H(skwGrad)	H(curl)	H(div)	L2
Vertex	$k = 1$				
Edge	$k = 2$	$k = 1$			
Planar	$k = 3$	$k = 2$	$k = 1$		
Facet	$k = 4$	$k = 3$	$k = 2$	$k = 1$	
Interior	$k = 5$	$k = 4$	$k = 3$	$k = 2$	$k = 1$

Degrees of Freedom H(grad)-Conforming

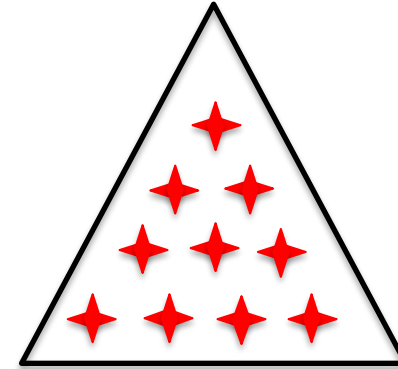
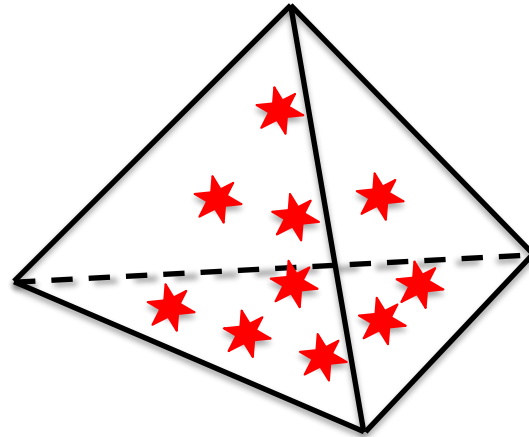
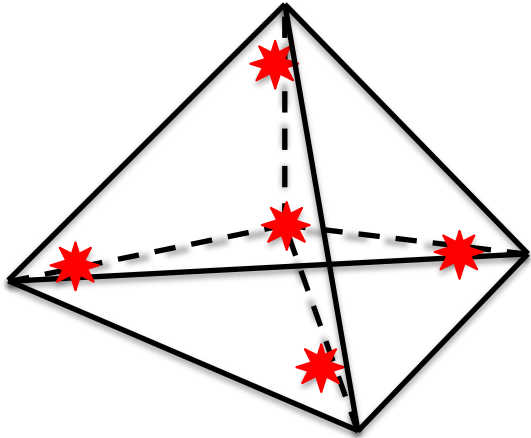
28



$$k = 6$$

Degrees of Freedom $H(\text{skwGrad})$ -Conforming

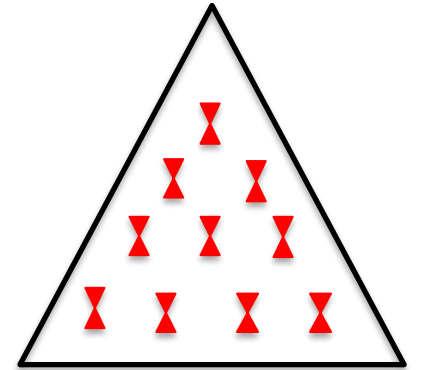
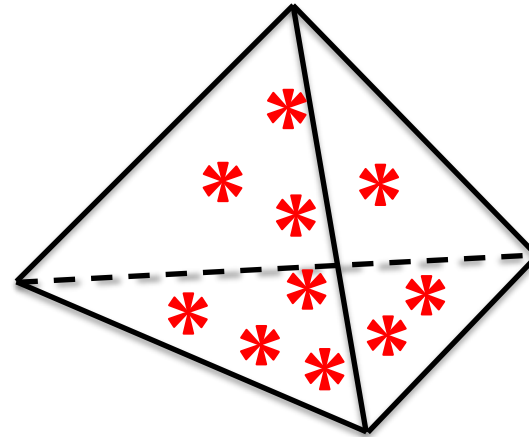
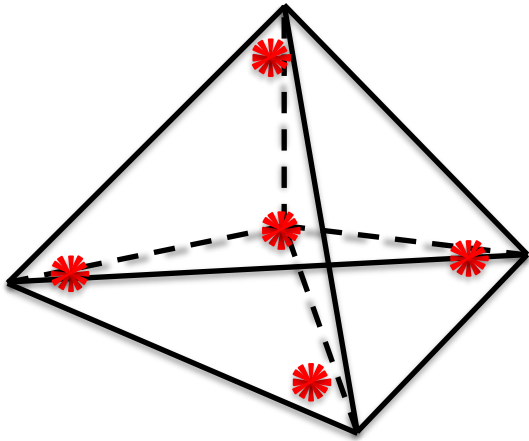
29



$k = 5$

Degrees of Freedom H(curl)-Conforming

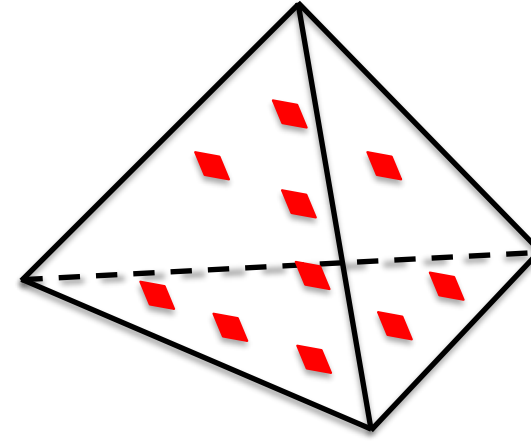
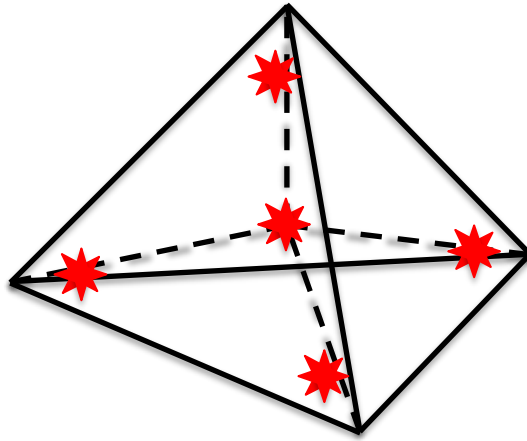
30



$$k = 4$$

Degrees of Freedom H(div)-Conforming

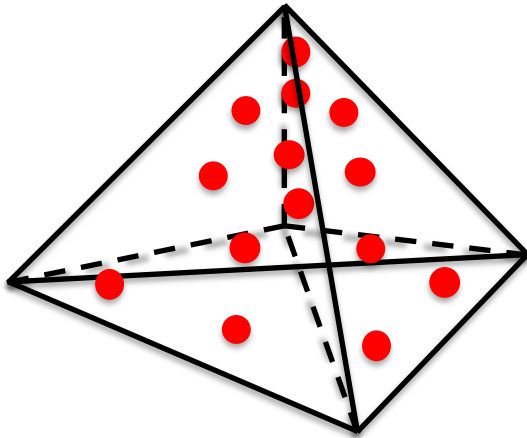
31



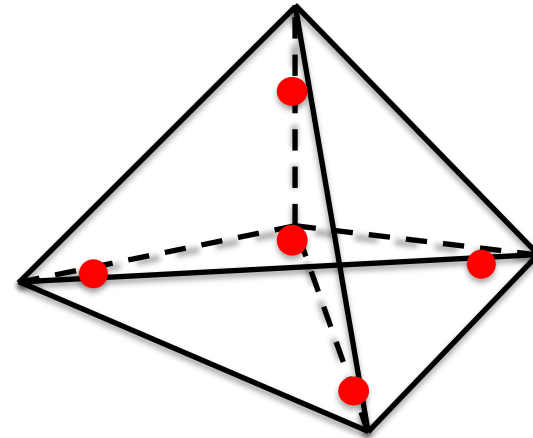
$k = 2$

Degrees of Freedom L2-Conforming

32



$k = 2$



$k = 1$

Constructing the Vandermonde Matrix (Scalar)

33

- A Vandermonde matrix is used in finite element method to transfer a modal basis, $\tilde{\psi}(\mathbf{r})$ into a nodal basis, $\mathbf{l}(\mathbf{r})$ representation
- To form the Vandermonde matrix the j^{th} scalar modal basis function, ψ_j is evaluated at points $\mathbf{r}_i$
- This operation returns a Vandermonde matrix, $\mathcal{V}_{ij}$

$$\mathcal{V}_{ij} = \psi_j(\mathbf{r}_i)$$

$$\mathcal{V}^T \mathbf{l}(\mathbf{r}) = \tilde{\psi}(\mathbf{r})$$

H(grad), L_2

Constructing the Vandermonde Matrix (Vector)

- Construct j^{th} vector basis function with $m = 0$ to 3 components, ψ_j^m evaluated at points $\mathbf{r}_i$
- Contract each function with the appropriate vector (depends on continuity condition of space)
 - H(skwgrad)-conforming elements enforce tangential continuity
 - H(div)-conforming elements enforce normal continuity
- This operation returns a generalized Vandermonde matrix $\mathcal{V}_{ij}$

$$\begin{aligned}\hat{\mathcal{V}}_{ijm} &= \psi_j^m(\mathbf{r}_i), \\ \mathcal{V}_{ij} &= \hat{\mathcal{V}}_{ijm} t_m.\end{aligned}$$

H(skwGrad)

$$\begin{aligned}\check{\mathcal{V}}_{ijm} &= \psi_j^m(\mathbf{r}_i), \\ \mathcal{V}_{ij} &= \check{\mathcal{V}}_{ijm} n_m.\end{aligned}$$

H(div)

Constructing the Vandermonde Matrix (Tensor)

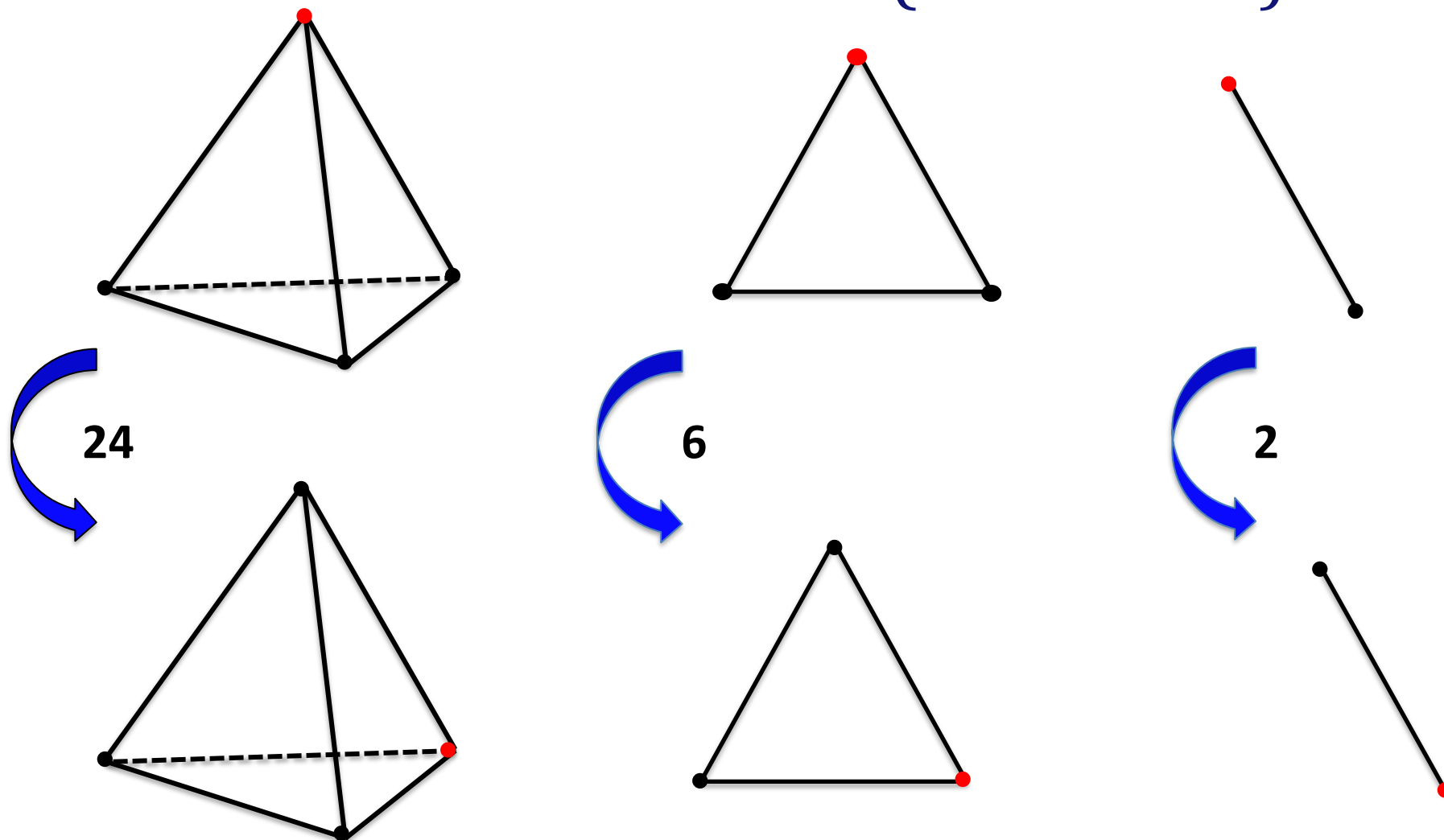
35

- Construct j^{th} tensor basis function with entries indexed by $m, l = 0, \dots, 3$ $\psi_j^{m,l}$ evaluated at points $\mathbf{r}_i$
- Contract each function with the appropriate vector pair t_m, t_l . (bivector)
- This operation returns a generalized Vandermonde matrix $\mathcal{V}_{ij}$

$$\hat{\mathcal{V}}_{ijml} = \psi_j^{m,l}(\mathbf{r}_i), \quad \mathcal{V}_{ij} = t_m \hat{\mathcal{V}}_{ijml} t_l$$

H(curl)

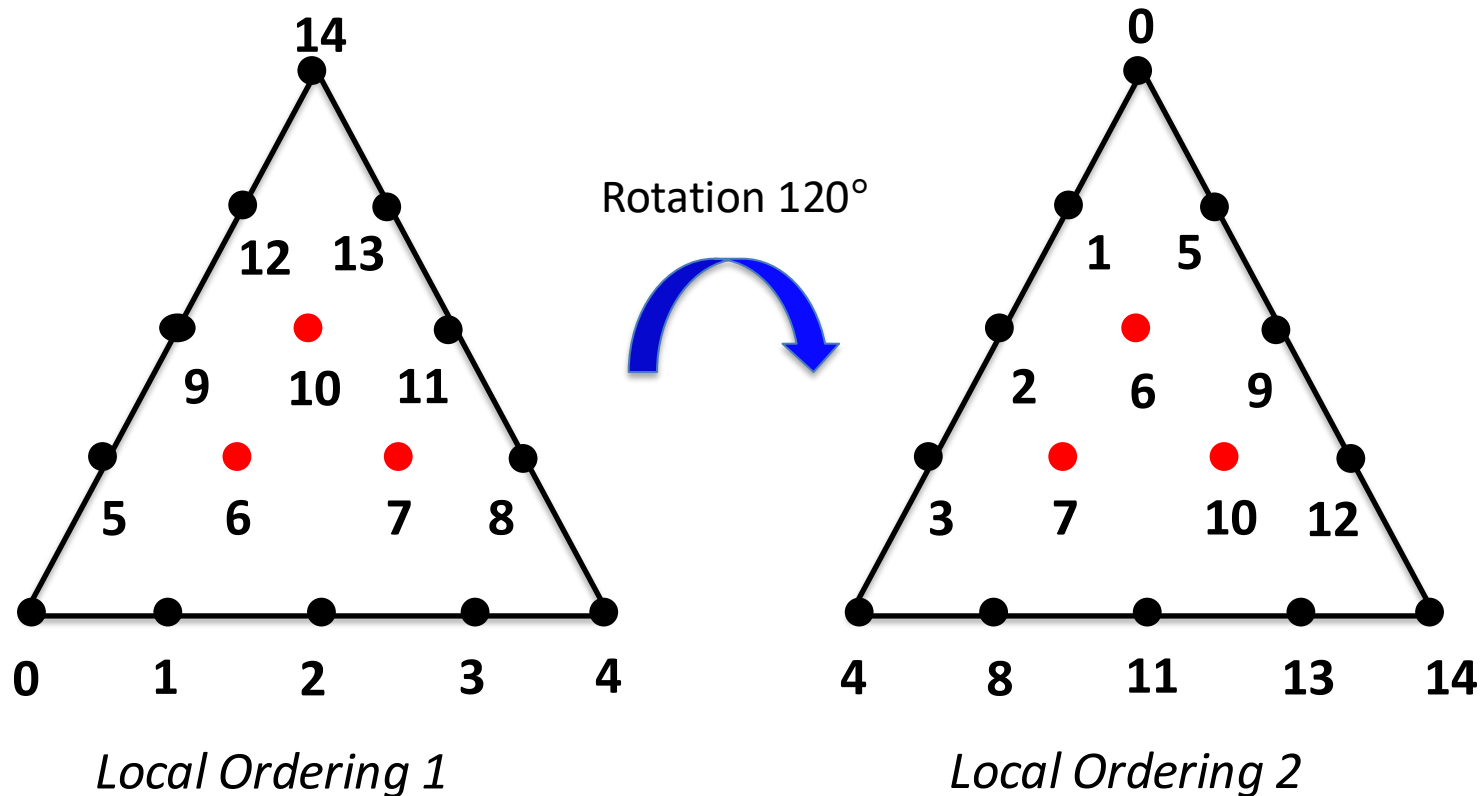
Orientation Based on Vertices (Geometric)



- Transformations: There are two types of transformations that create all possible orientations, *reflections* and *rotations*

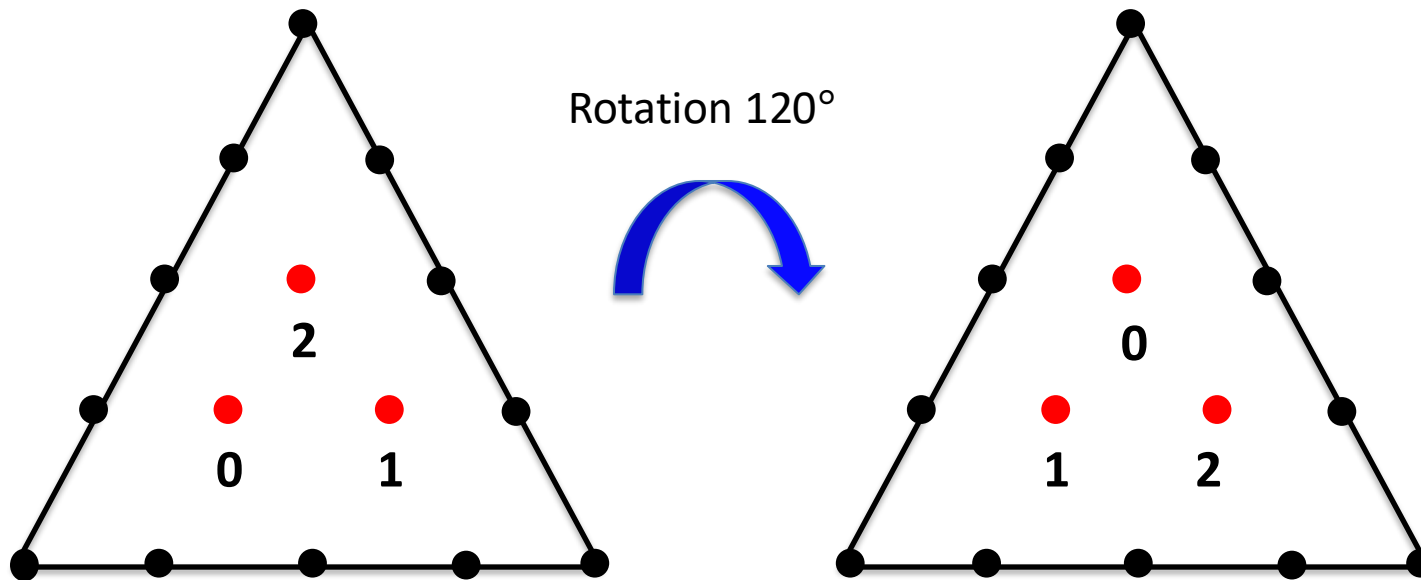
Degree of Freedom Considerations (Scalar Case)

- As k increases, how do we match **Location** DOFs on neighboring elements?
- Example: $k = 4$ Triangle Element



Reconcile Local Orderings of DOFs

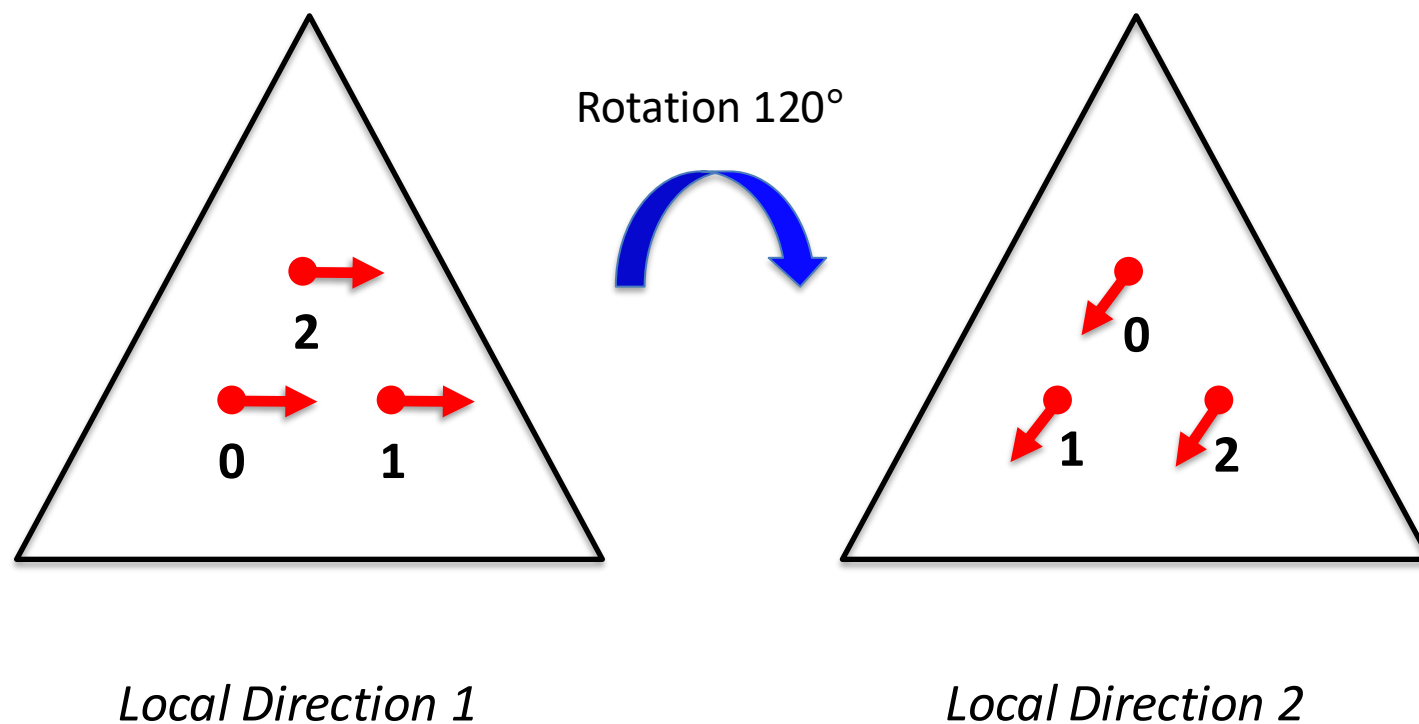
- Track orientations between neighboring elements by forming matrix that tracks different orderings of DOFs due to geometric transformations
- Only need to track DOFs that belong to geometric entity considered



Degree of Freedom Indexes		
0	1	2
1	0	2
2	0	1
2	1	0
1	2	0
0	2	1

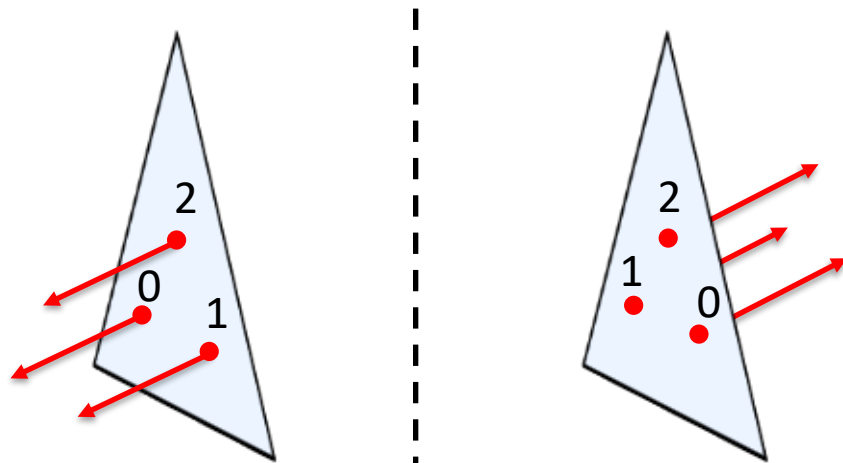
Degree of Freedom Considerations (Vector Case)

- As k increases, how do we match **Location** of DOFs on neighboring elements?
- As k increases, how do we match **Direction** of DOFs on neighboring elements?
- Example: Triangle Element



Orientation: H(div)-Conforming

- Normal vector direction is invariant under rotational transformations
- Reflectional transformations flip sign of normal vector



Reflection

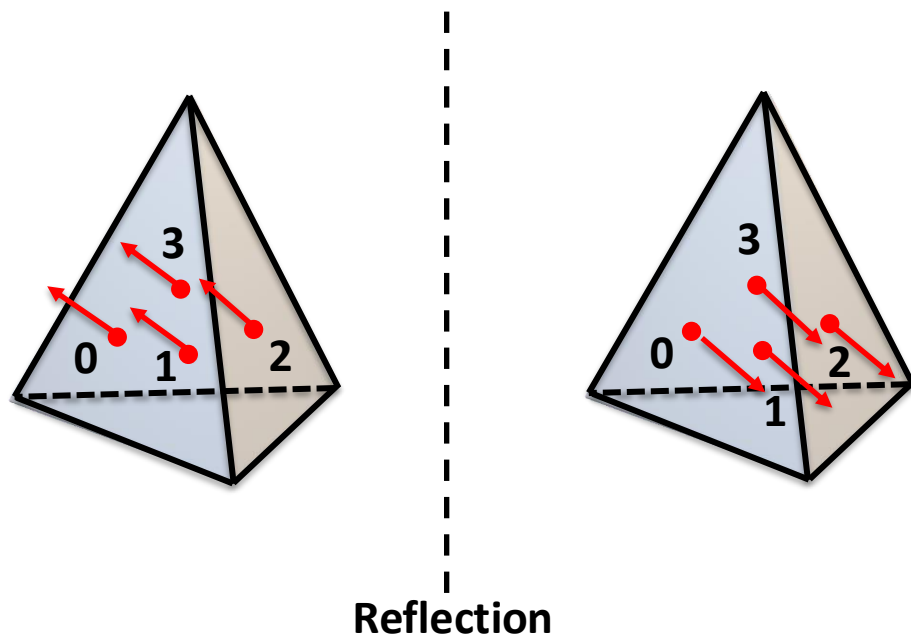
Normal Vector Sign Flip

Degree of Freedom Indexes		
0	1	2
1	0	2
2	0	1
2	1	0
1	2	0
0	2	1

Degree of Freedom Indexes		
0	1	2
-2	-1	-3
2	0	1
-3	-2	-1
1	2	0
-1	-3	-2

Orientation: H(div)-Conforming

- Normal vector direction is invariant under rotational transformations
- Reflectional transformations flip sign of normal vector



Normal Vector Sign Flip

Degree of Freedom Indexes			
0	1	2	3
0	1	3	2
0	3	1	2
0	3	2	1
2	0	1	3
3	0	1	2

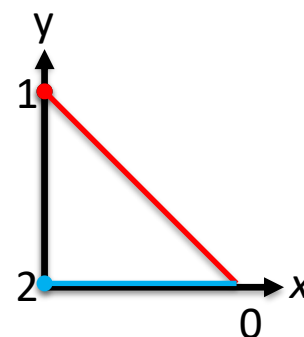
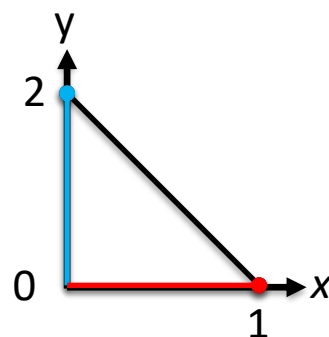
Degree of Freedom Indexes			
0	1	2	3
-1	-2	-4	-3
0	3	1	2
-1	-3	-2	-4
0	2	3	1
-1	-4	-3	-2

** Tables not complete

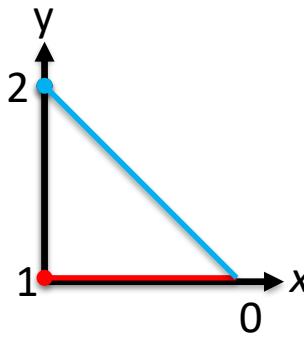
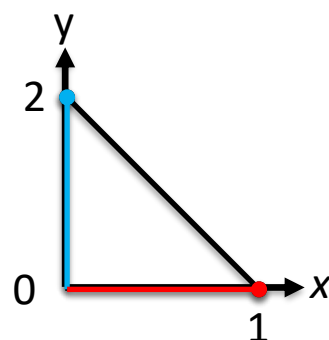
Orientation: H(skwGrad)-Conforming

- H(skwGrad)-conforming elements require ***tangential*** continuity between elements
- Requires a nontrivial linear transformation to track tangent vector directionality
- Current *ND_DofTransformation* class developed by M. Stowell

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$



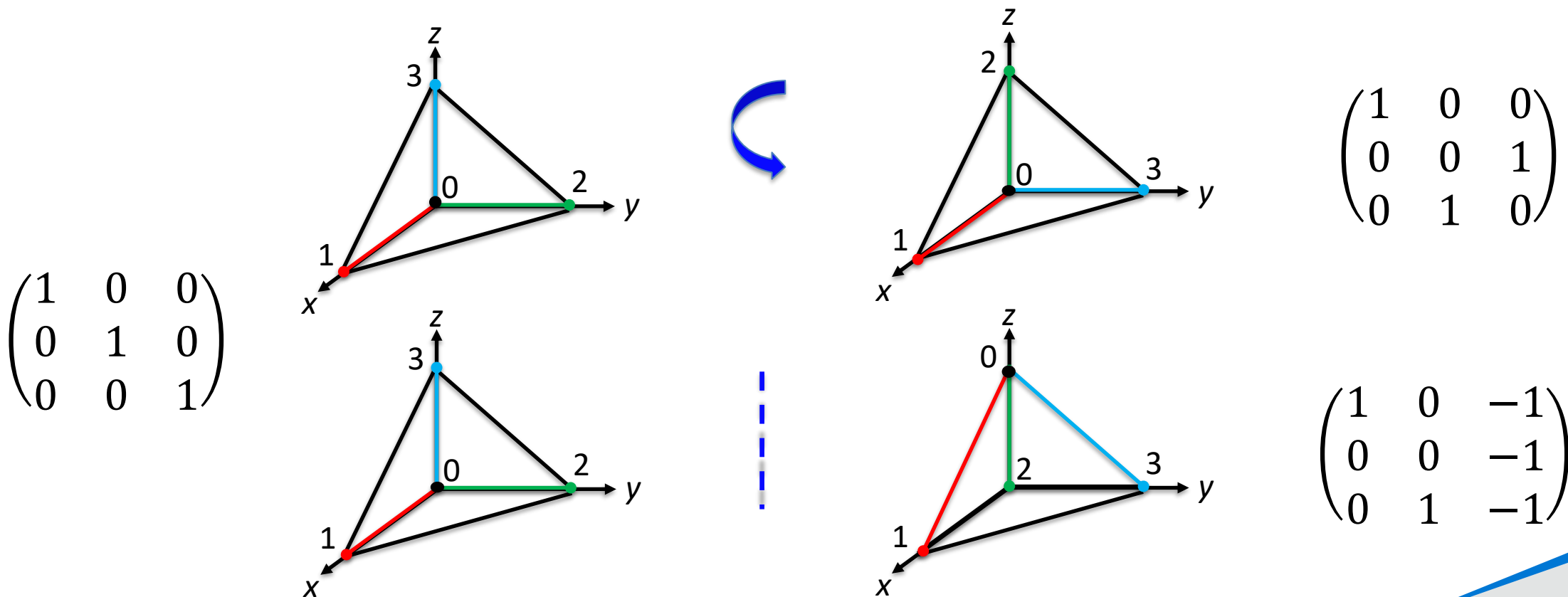
$$\begin{pmatrix} -1 & 1 \\ -1 & 0 \end{pmatrix}$$



$$\begin{pmatrix} -1 & 0 \\ -1 & 1 \end{pmatrix}$$

Orientation: H(skwGrad)-Conforming

- H(skwGrad)-conforming pentatopic elements require a nontrivial linear transformation for tetrahedral facets to enforce **tangential** continuity between elements
- This linear transformation is the natural extension of the triangle transformation

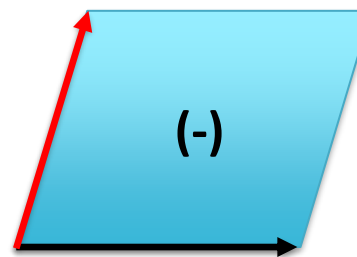
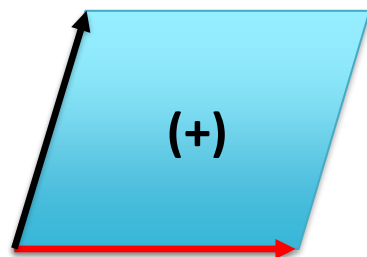


What is a Bivector?

- As we know, a vector has both a length and one direction
- A bivector is a wedge product of two vectors which itself is a skew-symmetric 2nd-order tensor

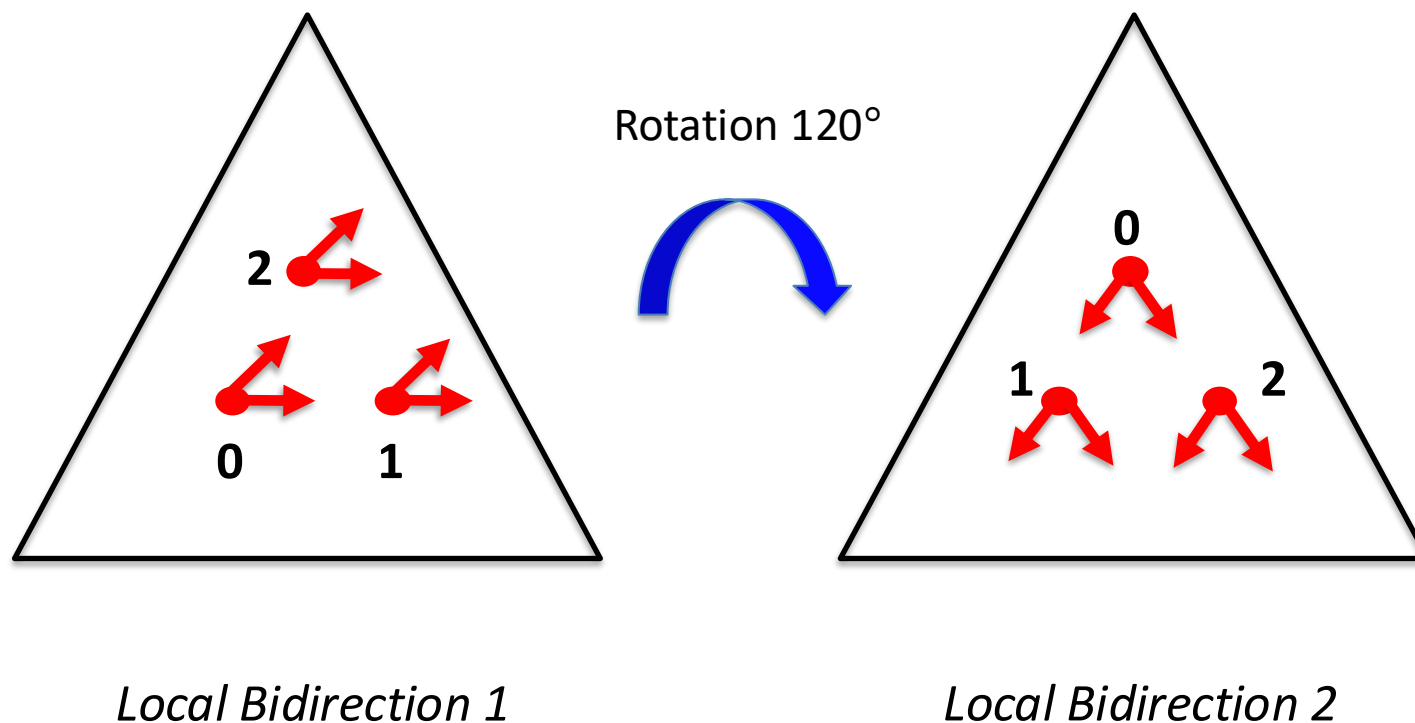
$$a \wedge b = a \otimes b - b \otimes a$$

- From a naïve perspective, a bivector has a signed area and two directions



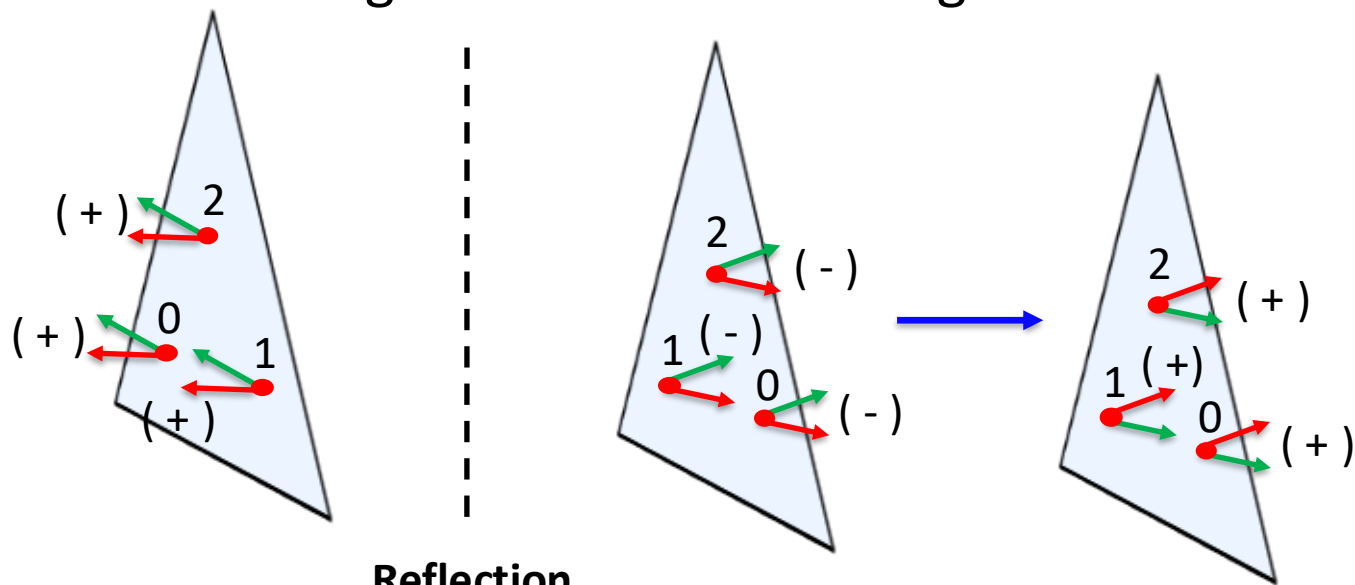
Degree of Freedom Considerations (Bivector Case)

- As k increases, how do we match **Location** of DOFs on neighboring elements?
- As k increases, how do we match **Bidirections** of DOFs on neighboring elements?
- Example: Triangle Element



Orientation: H(curl)-Conforming

- The two vectors that make up the bivector maintain the same angle and relative ordering under rotational transformations
- Reflectional transformations flip the relative order of the two vectors, while still maintaining the same relative angle



Reflection

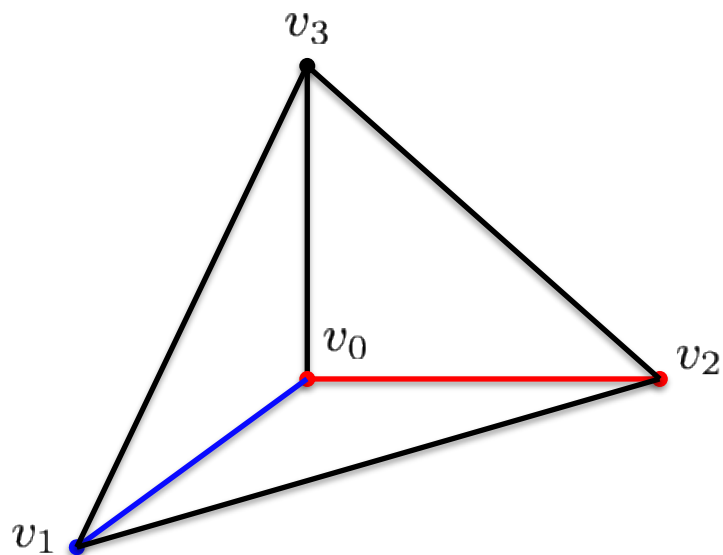
Bivector Sign Flip

Degree of Freedom Indexes		
0	1	2
1	0	2
2	0	1
2	1	0
1	2	0
0	2	1

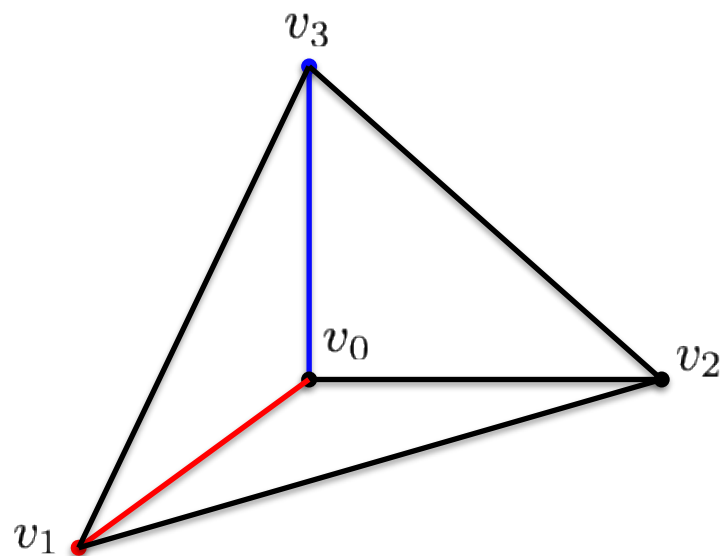
Degree of Freedom Indexes		
0	1	2
-2	-1	-3
2	0	1
-3	-2	-1
1	2	0
-1	-3	-2

Visualization of Bivectors on Tetrahedron

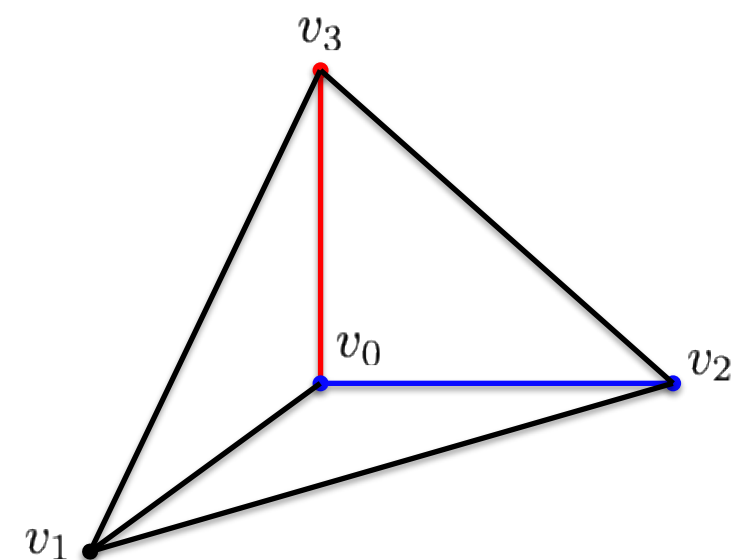
$$B_1 = (v_2 - v_0) \wedge (v_1 - v_0)$$



$$B_2 = (v_1 - v_0) \wedge (v_3 - v_0)$$

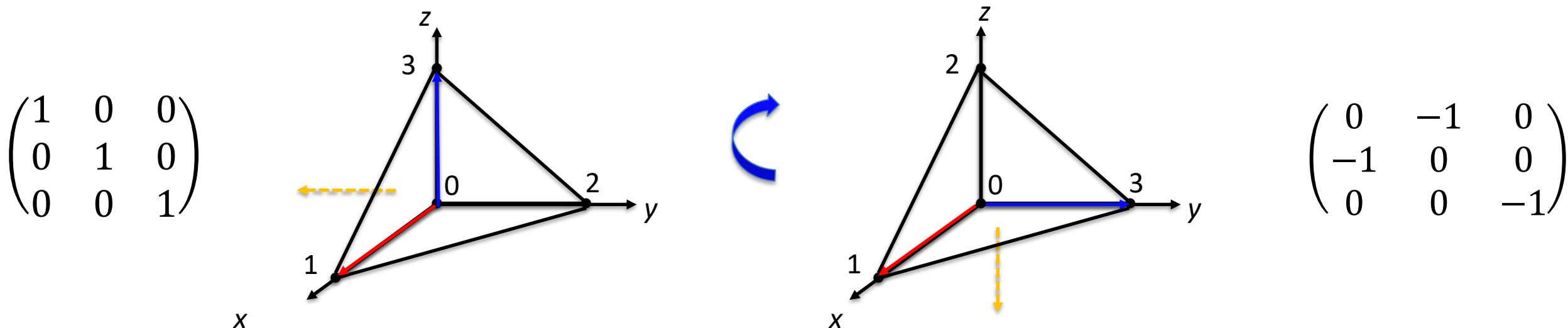


$$B_3 = (v_3 - v_0) \wedge (v_2 - v_0)$$



Orientation: H(curl)-Conforming

- H(curl)-conforming pentatopic elements require a nontrivial linear transformation for tetrahedral facets to enforce **bivector** continuity between elements
- This linear transformation is a new requirement for the 4D setting



IV. Results

Numerical Experiments

- We used Method of Manufactured Solutions to evaluate the order of accuracy of our finite element spaces
- We constructed a family of meshes by starting with the unit hypercube and performing a uniform subdivision
- Then on each hypercube we performed a Coxeter-Freudenthal-Kuhn (CFK) subdivision of the hypercube into 24 pentatopes

Order of Accuracy: H(grad)-Conforming

Problem:

$$-\Delta u = f$$

(Poisson Problem)

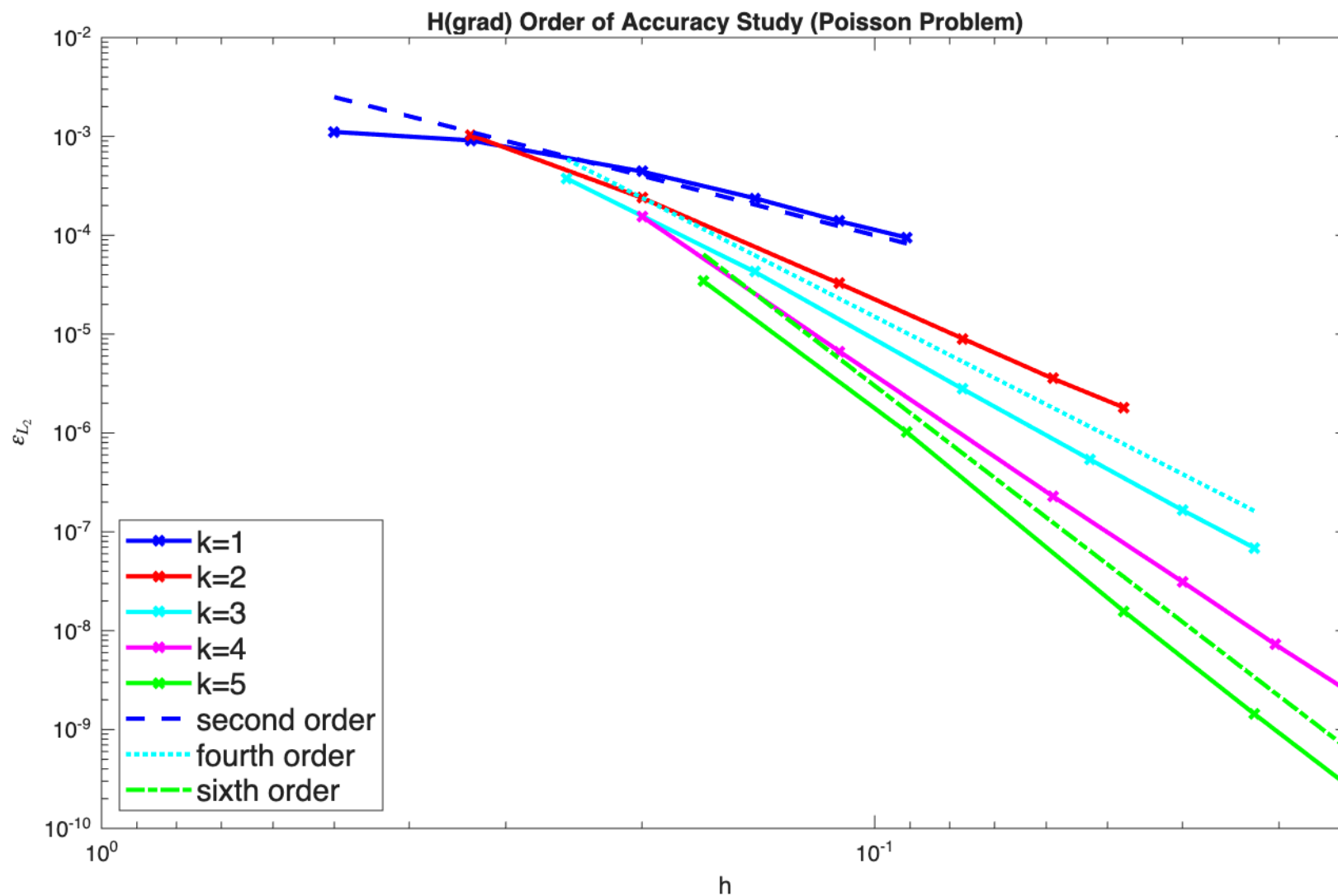
Solution:

$$u = x(1 - x)y(1 - y)z(1 - z)t(1 - t)$$

Meshes:

Mesh [# Elements]	$k = 1$ [# Dofs]	$k = 2$ [# Dofs]	$k = 3$ [# Dofs]	$k = 4$ [# Dofs]	$k = 5$ [# Dofs]
24	16	81	256	625	1,296
384	81	625	2,401	6,561	14,641
6144	625	6,561	28,561	83,521	194,481
31104	2,401	28,561	130,321	390,625	923,521
98236	6,561	83,487	390,421	1,185,241	2,824,061
239,996	14,641	194,479	923,509	2,825,721	

Order of Accuracy: H(grad)-Conforming



Order of Accuracy: H(skwGrad)-Conforming

Problem:

$$\text{Div}(\text{skwGrad}(G)) + G = f$$

(Div-skwGrad Problem)

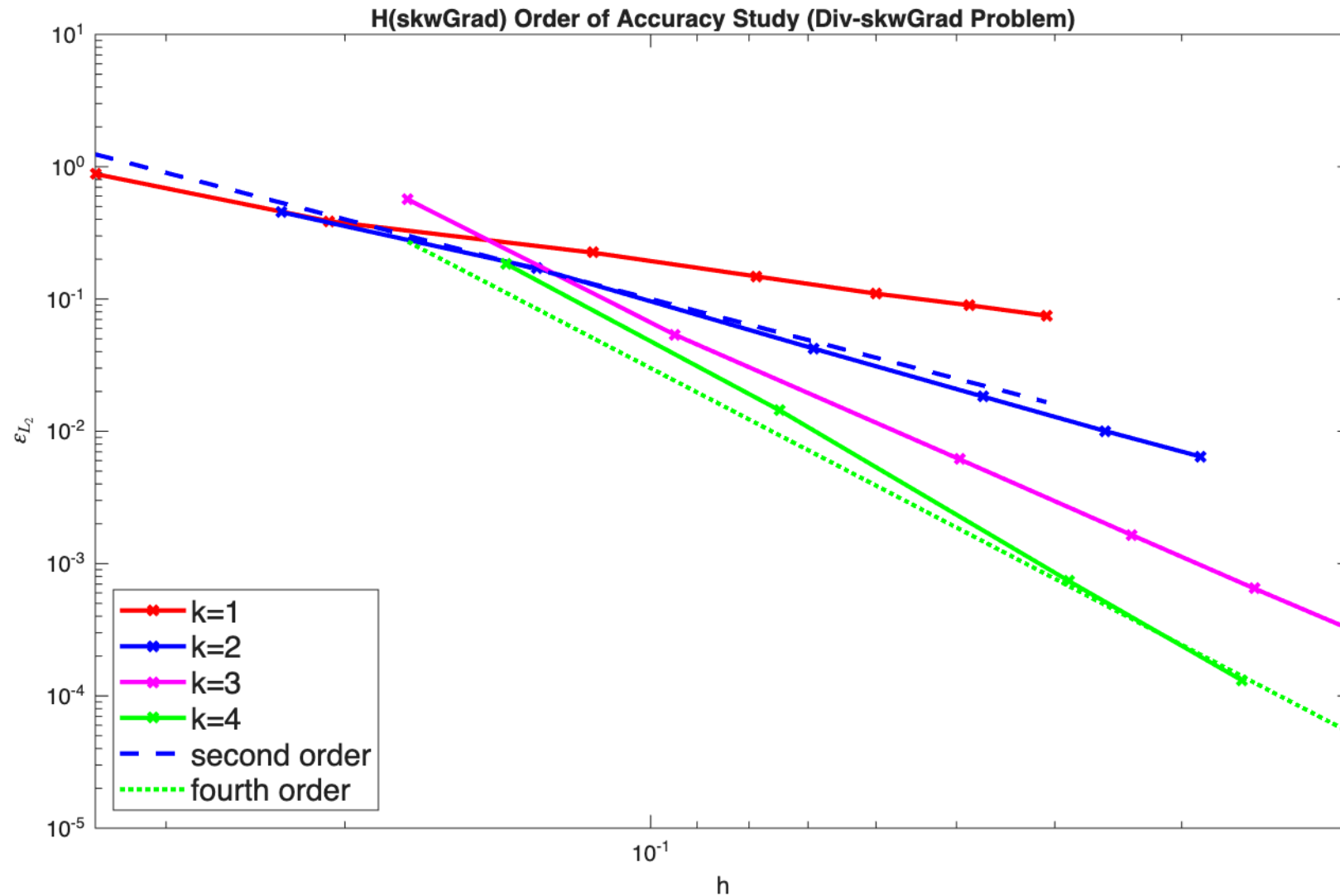
Solution:

$$G = \begin{bmatrix} \sin(\pi x) \cos(\pi y) \cos(\pi z) \cos(\pi t) \\ -\cos(\pi x) \sin(\pi y) \cos(\pi z) \cos(\pi t) \\ \cos(\pi x) \cos(\pi y) \sin(\pi z) \cos(\pi t) \\ -\cos(\pi x) \cos(\pi y) \cos(\pi z) \sin(\pi t) \end{bmatrix}$$

Meshes:

Mesh [# Elements]	$k = 1$ [# Dofs]	$k = 2$ [# Dofs]	$k = 3$ [# Dofs]	$k = 4$ [# Dofs]
24	65	350	1,107	2,684
384	544	3,552	12,480	32,320
6144	5,936	44,000	164,880	443,840
31104	26,160	203,520	780,912	2,131,584
98236	76,926	613,868		
239996	179,838	1,458,060		

Order of Accuracy: H(skwGrad)-Conforming



Order of Accuracy: H(curl)-Conforming

Problem:

$$\text{Curl}(\text{curl}(H)) + H = f$$

(Curl-curl Problem)

Solution:

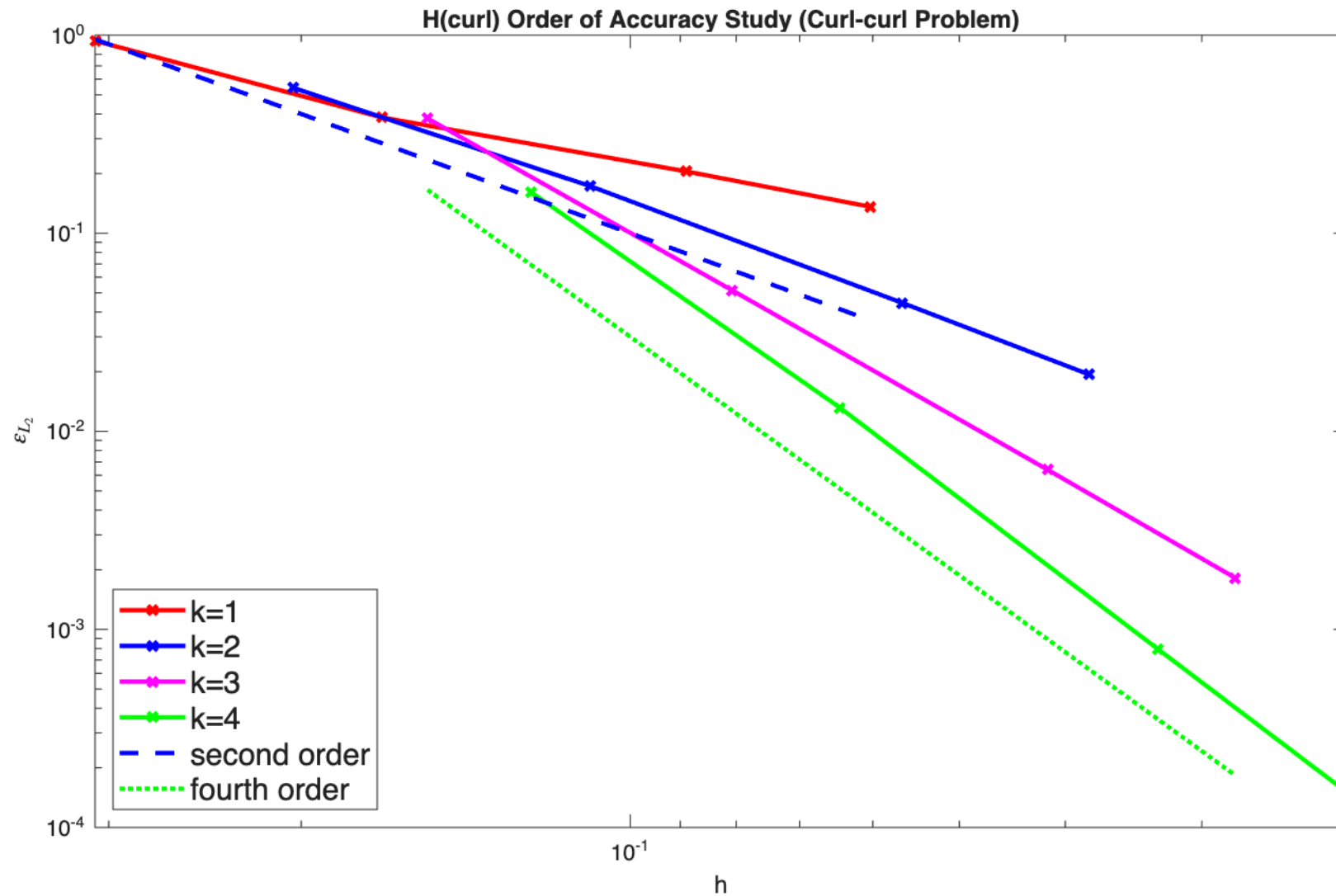
Let $s_i = \sin(\pi x_i)$ and $c_i = \cos(\pi x_i)$ for $x_i \in \{x, y, z, t\}$.

$$H = \frac{1}{2} \begin{bmatrix} 0 & s_x s_y c_z c_t & s_x c_y s_z c_t & s_x c_y c_z s_t \\ -s_x s_y c_z c_t & 0 & c_x s_y s_z c_t & c_x s_y c_z s_t \\ -s_x c_y s_z c_t & -c_x s_y s_z c_t & 0 & c_x c_y s_z s_t \\ -s_x c_y c_z s_t & -c_x s_y c_z s_t & -c_x c_y s_z s_t & 0 \end{bmatrix}$$

Meshes:

Mesh [# Elements]	$k = 1$ [# Dofs]	$k = 2$ [# Dofs]	$k = 3$ [# Dofs]	$k = 4$ [# Dofs]
24	110	582	1,812	4,340
384	1,232	7,152	23,520	58,400
6144	16,064	98,880	336,000	851,840
31104	75,600	475,632	1,635,552	4,177,440

Order of Accuracy: H(curl)-Conforming



Order of Accuracy: H(div)-Conforming

Problem:

$$-\nabla(\nabla \cdot G) + G = f$$

(Grad-div Problem)

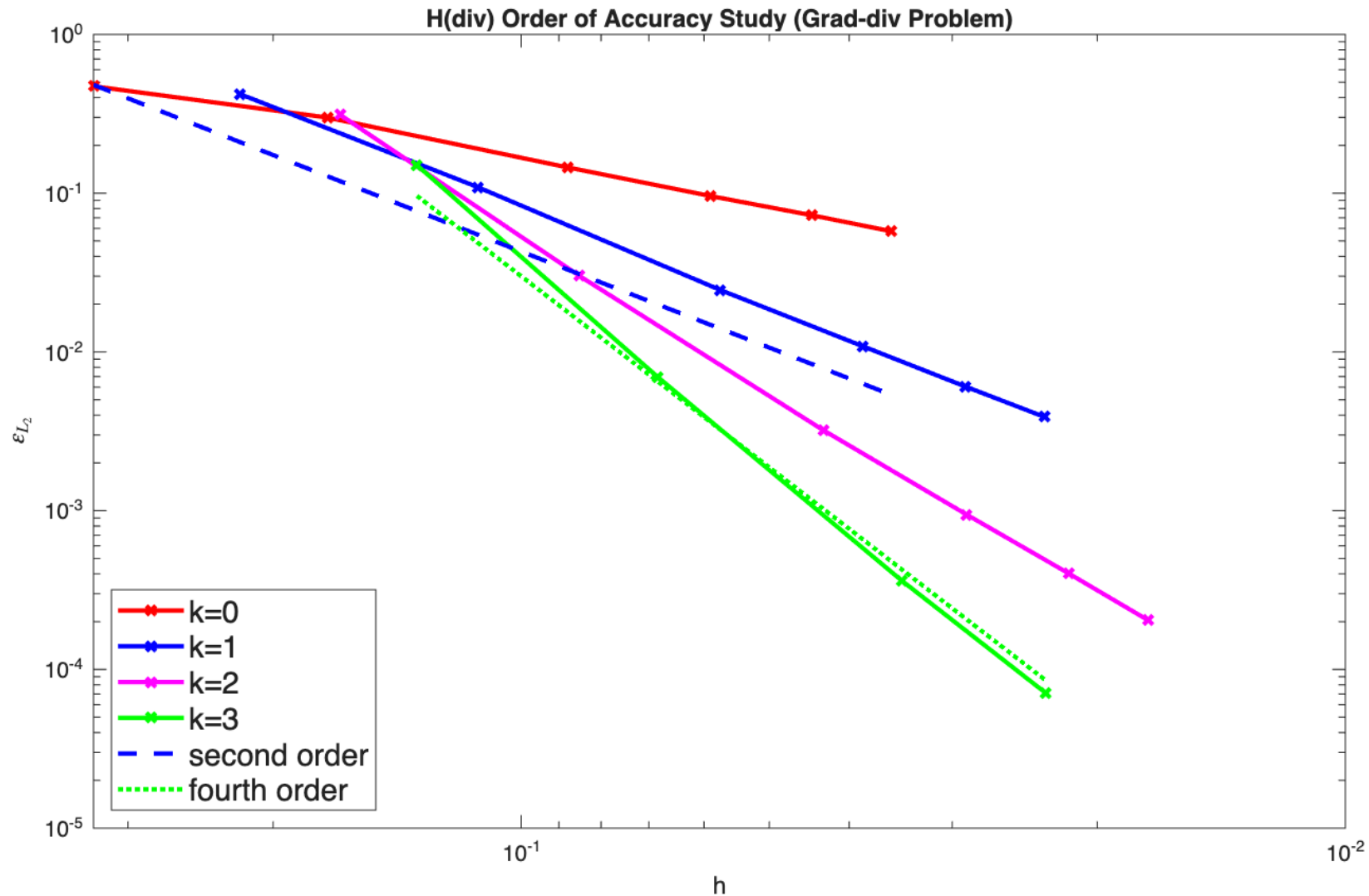
Solution:

$$G = \begin{bmatrix} \cos(\pi x) \sin(\pi y) \sin(\pi z) \sin(\pi t) \\ \cos(\pi y) \sin(\pi z) \sin(\pi t) \sin(\pi x) \\ \cos(\pi z) \sin(\pi t) \sin(\pi x) \sin(\pi y) \\ \cos(\pi t) \sin(\pi x) \sin(\pi y) \sin(\pi z) \end{bmatrix}$$

Meshes:

Mesh [# Elements]	$k = 0$ [# Dofs]	$k = 1$ [# Dofs]	$k = 2$ [# Dofs]	$k = 3$ [# Dofs]
24	84	432	1,320	3,120
384	1,152	6,144	19,200	46,080
6144	16,896	92,160	291,840	706,560
31104	82,944	456,192	1,451,520	3,525,120
98236	257878	1424456	4543500	
239996	623990	3455944	11039820	

Order of Accuracy: H(div)-Conforming



Order of Accuracy: L2-Conforming

Problem:

$$-\Delta u = f$$

(Poisson Problem)

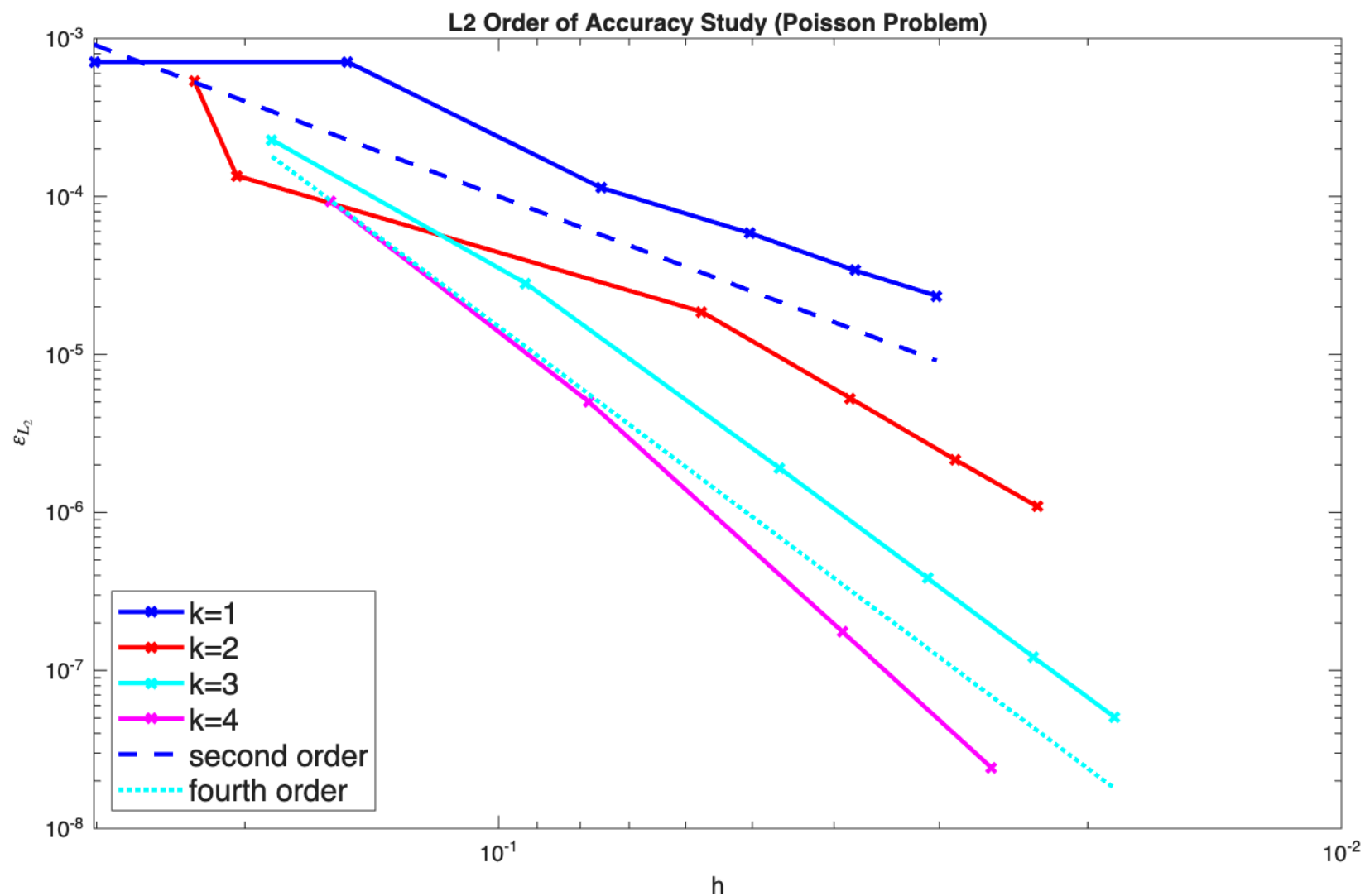
Solution:

$$u = x(1 - x)y(1 - y)z(1 - z)t(1 - t)$$

Meshes:

Mesh [# Elements]	$k = 1$ [# Dofs]	$k = 2$ [# Dofs]	$k = 3$ [# Dofs]	$k = 4$ [# Dofs]
24	120	360	840	1,600
384	1,920	5,760	13,440	26,880
6144	30,720	92,160	215,040	430,080
31104	155,520	466,560	1,088,640	2,177,280
98236	491,180	1,473,540		
239996	1,199,980	3,599,940		

Order of Accuracy: L2



V. Final Thoughts

Future Work

- Merge current 4D capabilities into official MFEM: 4d_dev branch
- Merge updated MFEM: 4d_dev branch into main MFEM version

